STARWEAVER

Hands-On-Learning: Secure Coding Practices: Identifying and Fixing Vulnerable Code in GitHub Codespaces

Learning Objective

By the end of this activity, learners will be able to:

- Identify common security vulnerabilities in application code including input validation flaws, output encoding issues, and OWASP Top 10 vulnerabilities
- Apply secure coding practices to fix broken access control, injection attacks, and cryptographic failures
- Implement proper input validation and output encoding techniques

- Use GitHub Codespaces effectively for secure development practices
- Test and validate security fixes using automated tools and manual testing
- Document security improvements and share findings through peer review

Description

This focused hands-on lab provides learners with a quick, practical experience in identifying and fixing a specific security vulnerability using GitHub's Secure Code Game. Students will work with one intentionally vulnerable code challenge to apply secure coding practices and validate their fix, emphasizing how security can be built into applications from the start.

Structure of the Activity

Tools used

- GitHub Codespaces
- GitHub Secure Code Game repository
- Visual Studio Code (web-based)

Scenario

You are a security-focused developer working for a technology company that has recently experienced several security incidents. The organization has decided to implement a comprehensive secure coding training program to prevent future vulnerabilities. As part of this initiative, you've been tasked with working through a series of intentionally vulnerable code challenges that mirror real-world security flaws found in production applications. Your goal is to identify these vulnerabilities, understand their potential impact, and implement secure fixes that follow OWASP best practices. This exercise will help you develop the skills needed to write secure code from the start and contribute to your organization's security-first development culture.

Objective

Fix one intentionally vulnerable code challenge (Season 1, Level 1) by applying secure coding principles including input validation and proper error handling.

Instructions for learners

1. Set up GitHub Codespaces Environment

- Navigate to the GitHub Secure Code Game repository: https://github.com/skills/secure-code-game
- Click the green "Use this template" button to create your own copy of the repository
- In your new repository, click the "Code" dropdown button and select "Create codespace on main"

- Wait for the codespace to initialize (this may take 2-3 minutes)
- Once loaded, navigate directly to the Season-1/Level-1 folder

2 Analyze the Vulnerable Code

- Open the Season-1/Level-1 folder and read the README.md file
- Examine the vulnerable Python code file
- Identify the security vulnerability (typically an input validation issue)
- Run the code using the terminal to observe the vulnerable behavior
- Note how malicious input can exploit the application

3 Implement Security Fix

- Apply proper input validation to sanitize user input
- Implement secure error handling that doesn't expose sensitive information
- Follow the hints provided in the challenge README
- Test your fix with both normal and malicious input
- Ensure the application still functions correctly with valid input

4 Validate and Test Your Fix

- Run the automated validation provided in the repository
- Test edge cases and confirm the vulnerability is resolved
- Verify that error messages are appropriately handled
- Commit your changes with a descriptive message

5 Document Your Solution

- Take a screenshot of your successful fix
- Write a brief summary of the vulnerability found and how you fixed it
- Note the secure coding principle applied (e.g., input validation, sanitization)
- Prepare for peer review discussion

Organize Information Following This Structure for Peer Review

1. Title Page:

Project title: "Secure Coding Lab: Input Validation Vulnerability Fix" Your name, date, level completed (Season 1, Level 1)

Requirement 1: Vulnerability Identification

- Describe the specific vulnerability found in the code
- Classify the vulnerability (e.g., improper input validation, CWE-20)
- Explain how this vulnerability could be exploited by an attacker

Requirement 2: Security Fix Implementation

- Show before/after code comparison
- Explain the secure coding principle applied (input validation/sanitization)
- Describe how the fix prevents the exploitation

Instructions for Documenting and Sharing the Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Insert the diagram, map, screenshot, or image directly into your Word document, if available.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

Share the Project

- Save your report and presentation in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.

Additional Resources and References:

- OWASP Secure Coding Practices Guide: https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/
- OWASP Top 10 2021: https://owasp.org/Top10/
- OWASP Input Validation Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- GitHub Codespaces Documentation: https://docs.github.com/en/codespaces
- OWASP Proactive Controls: https://top10proactive.owasp.org/